



Smart Edge Environmental Monitoring System

A real-time environmental monitoring system built on **ESP32 + DHT11**, using **MQTT** for lightweight edge-to-cloud communication and a **Streamlit** dashboard for live visualization.

Developed as part of a Summer Research Internship at **IIIT Vadodara International**

Campus, in the domain of Telco Cloud, Edge Computing, IoT Integration, and the Computing Continuum.

The ESP32 acts as an edge device — sensing and validating data locally before publishing it — while MQTT provides a lightweight publish/subscribe transport suited to constrained edge hardware, and the dashboard renders live metrics, status, and alerts.



Table of Contents

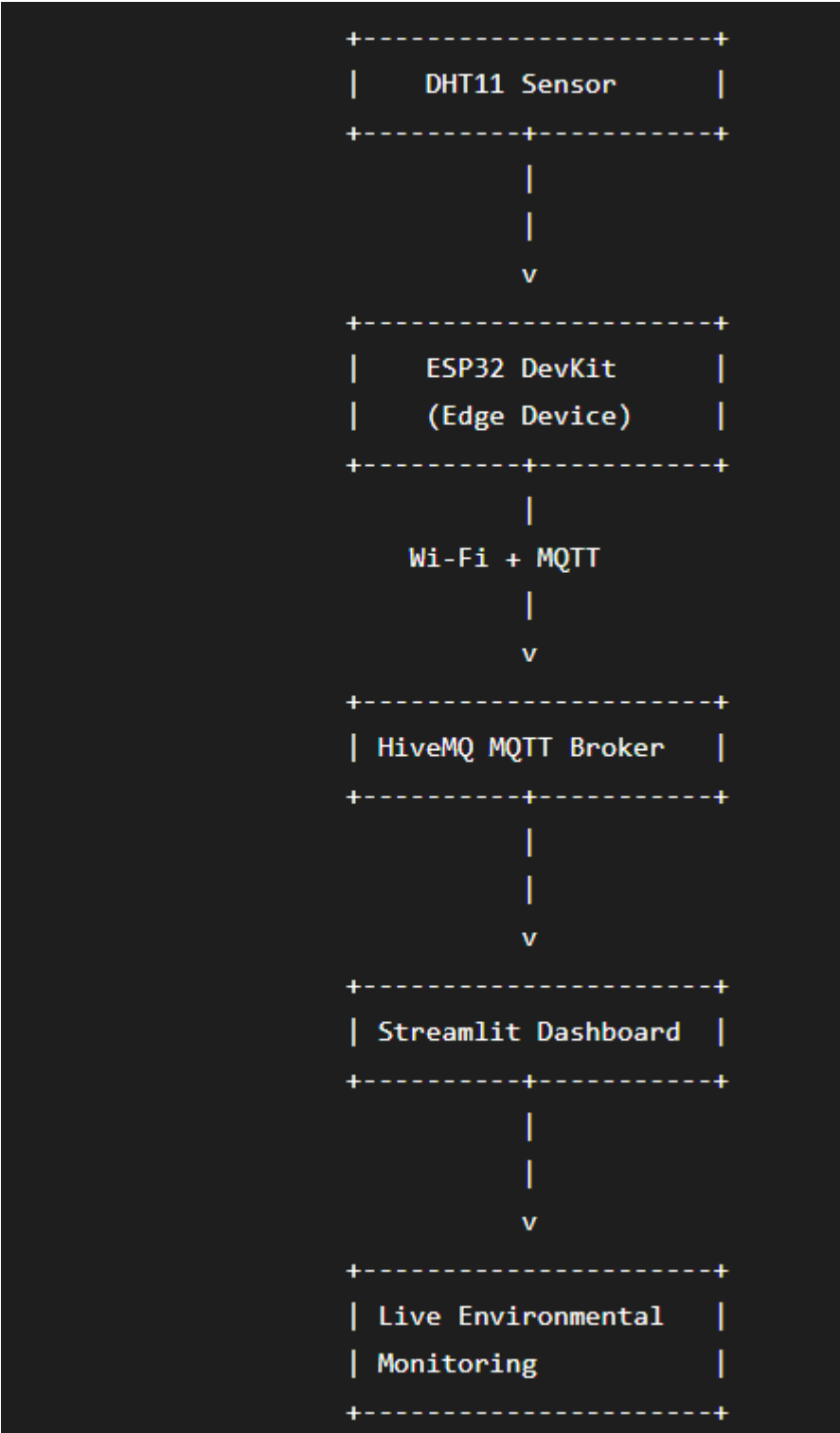
- [Features](#)
 - [System Architecture](#)
 - [Hardware Requirements](#)
 - [Software Requirements Folder](#)
 - [Structure](#)
 - [Installation & Setup](#)
 - [MQTT Topics](#)
 - [Screenshots](#)
 - [Future Improvements](#)
 - [License](#)
 - [Author](#)
-



Features

- Real-time temperature and humidity sensing using a DHT11 sensor on an ESP32
 - Local (edge-side) data validation before transmission, discarding invalid sensor reads
 - Lightweight MQTT publish/subscribe communication over Wi-Fi
 - Automatic MQTT reconnection on connection loss Live Streamlit
 - dashboard with:
 - Real-time temperature and humidity metrics
 - Device online/offline status indicator
 - Rolling trend line chart (last 50 readings)
 - Threshold-based alerts (high temperature, rising temperature, low/high humidity)
-

 System Architecture



The ESP32 performs sensing and preprocessing directly at the edge, and MQTT provides a lightweight, decoupled communication layer between the edge device and the visualization layer — keeping this project grounded in edge computing principles rather than a simple cloud-only IoT setup.

 Hardware Requirements

Component	Purpose
ESP32 Development Board	Wi-Fi-enabled microcontroller (edge device)

DHT11 Sensor	Digital temperature and humidity sensor
Breadboard	Prototyping platform
Jumper Wires	Sensor-to-ESP32 wiring
USB Cable	Power and serial programming

Wiring: DHT11 data pin → ESP32 **GPIO 4**

Software Requirements

- PlatformIO (VS Code extension or CLI)
- Arduino framework for ESP32 (espressif32 platform)
- Python 3.8+
- Python package s: `streamlit`, `paho-mqtt`, `pandas`
- Arduino libraries (inst alled a utom aticall y via `platformio.ini`):
 - `Adafruit DHT sensor library`
 - `PubSubClient`

Folder Structure

```
Smart-Edge-Environmental-Monitoring-System/
├── src/
│   └── main.cpp           # ESP32 firmware
├── Screenshots/
│   ├── Hardware_Setup.jpg
│   ├── Serial_Monitor.png
│   ├── Dashboard.png
│   ├── Live_Graph.png
│   └── Complete_System.jpg
├── dashboard.py           # Streamlit Dashboard
├── Subscriber.py          # MQTT Subscriber
├── platformio.ini         # PlatformIO Configuration
├── requirements.txt       # Python Dependencies
├── README.md
├── LICENSE
└── .gitignore
```



Installation & Setup

1. Clone the repository

```
Cpp
const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
```

3. Wire the hardware

```
bash
pio run --target upload
```

Connect the DHT11 sensor's data pin to **GPIO 4** on the ESP32, and power the sensor from the ESP32's 3.3V/5V and GND pins as per your sensor module's specification.

4. Build and upload the firmware Using

PlatformIO CLI:

Or open the project folder in VS Code with the PlatformIO extension and use **Upload**.

5. Monitor serial output (optional)

```
bash
pio device monitor
```

You should see Wi-Fi and MQTT connection logs followed by periodic temperature/humidity readings.

6. Install Python dependencies

7. Run the dashboard

```
bash
pip install -r requirements.txt
```

The dashboard will open in your browser and begin displaying live data once the ESP32 starts publishing.



MQTT Topics

Topic	Payload	Published By
<code>iiitv/temperature</code>	Temperature in °C (string)	ESP 32
<code>iiitv/humidity</code>	Humidity in % (string)	ESP 32

Broker used: `broker.hivemq.com` (public HiveMQ broker, port 1883)

This project uses the public HiveMQ broker **only for prototype demonstration**. It provides no authentication or encryption. For any production or long-term deployment, use a private, secured MQTT broker with TLS and access control.



Screenshots

Hardware Setup `screenshots/01_hardware_setup.jpg`

Serial Monitor Output `screenshots/02_serial_monitor.png`

Dashboard — Main View `screenshots/03_dashboard_main.png`

Dashboard — Live Trend Graph & Alert `screenshots/04_live_graph.png`

(Add your screenshot files to the `screenshots/` folder using the filenames above so they render correctly on GitHub.)



Future Improvements

- ☐ • Migrate from the public HiveMQ broker to a secure, private MQTT broker with TLS and authentication
- ☐ • Re-enable and extend the dashboard statistics panel (average/maximum/minimum), which is currently present in the code but disabled
- ☐ • Add payload validation on the dashboard to handle malformed or missing MQTT messages
- ☐ • Add local data logging or edge-level aggregation on the ESP32 to reduce dependence on continuous connectivity
- ☐ • Expand the sensor suite (e.g., air quality, light) within the same architecture



License

This project is licensed under the [MIT License](#).



KUNA VARSHITH KUMAR Summer Research Intern — IIIT Vadodara International Campus Domain:

Telco Cloud • Edge Computing • IoT Integration • Computing Continuum

[Github](#) • [LinkedIn](#)